L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #5 Due: 2025-05-06

Learning Objectives

In this assignment you will demonstrate:

- ✅ The ability to write regular expressions
- ✅ The ability to perform a problem reduction to demonstrate undecidability
- ✅ An understanding of the major optimizations from compiler history (especially the Frances Allen ones)
- ✅ The ability to write optimizations in a compiler
- ✅ Excellence in growing and maintaining a large Node.js project

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings from the first four assignments that you did not get to.
- ✅ Work through the course notes on How to Write a Compiler, Regular Expressions, Intermediate Representations, and Code Optimization.
- ✅ Read Ch. 9 in [Bernhardt]
- ✅ Read Ch. 7 of [Sipser], and skim Ch. 8-10
- ✅ Skim Ch. 10-12 of [Mogenson] (paying attention 11-12)
- ✅ Browse the vast amount of material at regular-expressions.info. Play around with Regex 101 and Regex Golf. Mastering regular expressions is something that should be expected of you at this stage of your career. To see some examples of regexes in the wild, check out this overview of credit card validation.

- ✅ Browse the Wikipedia article on Program Optimization
- ✅ Study this "Optimization Killers" article. You'll be glad you did. There are so many good things in here you can impress your friends (and future interviewers) with!
- ✅ Browse this great presentation by Graydon Hoare. You don't have to understand every slide, but jot down notes from some of the big ideas. If it gets too technical, don't worry, just move on.
- ✅ Read Frances Allen's classic A Catalogue of Optimizing Transformations. This work followed two of her previous papers where she created new frameworks and algorithms and structures for performing and implementing static program analysis. You may also wish to read this remembrance from IBM (where she was a fellow), or any number of bibliographies on her life and work.

The following readings are optional in the sense that no direct assessment will be based on them. However, as this course marks your deepest dive yet into computing theory and history, you'd be wise to familiarize yourself with the following:

- ✅ Alan Turing's Forgotten Ideas
- ✅ Ada and the First Computer
- ✅ Decoding an Ancient Computer
- ✅ The Origins of Computing

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. Write JavaScript regular expressions for the following. Please take advantage of character classes, lookarounds, and backreferences where they apply.
   a. Canadian Postal Codes (make sure to prohibit D, F, I, O, Q, U)
   b. Legal Visa® Card Numbers, *ignoring the Luhn checksums*, i.e., accept `4` + 15 digits or `4` + 12 digits.
   c. Legal MasterCard® Numbers, *ignoring the Luhn checksums*, i.e., accept `51–55` + 14 digits or `2221–2720` + 12 digits.
   d. Strings of Basic Latin letters **except** those strings that are exactly three letters ending with two Latin letter o's, of any case.
   e. Binary numerals divisible by 16.
   f. Decimal numerals in the range 8 through 32, inclusive.
   g. All strings of Unicode letters, except `python`, `pycharm`, or `pyc`.
   h. Floating point constants that are allowed to have an empty fractional part, but whose exponent part is required and can have no more than three digits in the exponent part
   i. Palindromes over the letters a, b, and c, of length 2, 3, 5, or 8
   j. Python string literals. Don't get too fancy here—just translate what you see in the Python Reference linked above into Ohm notation.

   Structure your application and its unit tests as follows. The test file *test/regex_exercises.test.js* should be:

```
import { describe, it } from "node:test"
import assert from "assert"
import { matches } from "../regex_exercises.js"

const testFixture = {
  canadianPostalCode: {
    good: ["A7X 2P8", "P8E 4R2", "K1V 9P2", "Y3J 5C0"],
    bad: ["A7X   9B2", "C7E 9U2", "", "Dog", "K1V\t9P2", " A7X 2P8", "A7X 2P8 "]
  },
  visa: {
    good: ["4128976567772613", "4089655522138888", "4098562516243"],
    bad: [
      "43333",
      "423462387462837468 23",
      "7687777777263211",
      "foo",
      "π",
      "4128976567772613 ",
```

```
  ],
},
masterCard: {
  good: [
    "5100000000000000",
    "5294837679998888",
    "5309888182838282",
    "5599999999999999",
    "2221000000000000",
    "2720999999999999",
    "2578930481258783",
    "2230000000000000",
  ],
  bad: [
    "5763777373890002",
    "513988843211541",
    "51398884321108541",
    "",
    "OH",
    "5432333xxxxxxxxx",
  ],
},
notThreeEndingInOO: {
  good: ["", "fog", "Tho", "one", "a", "ab", "food"],
  bad: ["fOo", "gOO", "HoO", "zoo", "MOO", "123", "A15"],
},
divisibleBy16: {
  good: ["0", "00", "000", "00000", "00000", "000000", "00000000", "1101000000
  bad: ["1", "00000000100", "1000000001", "dog0000000"],
},
eightThroughThirtyTwo: {
  good: Array(25)
    .fill(0)
    .map((x, i) => i + 8),
  bad: ["1", "0", "00003", "dog", "", "361", "90", "7", "-11"],
},
notPythonPycharmPyc: {
  good: ["", "pythons", "pycs", "PYC", "apycharm", "zpyc", "dog", "pythonpyc"]
  bad: ["python", "pycharm", "pyc"],
},
restrictedFloats: {
```

```
  good: ["1e0", "235e9", "1.0e1", "1.0e+122", "55e20"],
  bad: ["3.5E9999", "2.355e-9991", "1e2210"],
},
palindromes2358: {
  good: [
    "aa",
    "bb",
    "cc",
    "aaa",
    "aba",
    "aca",
    "bab",
    "bbb",
    "ababa",
    "abcba",
    "aaaaaaaa",
    "abaaaaba",
    "cbcbbcbc",
    "caaaaaac",
  ],
  bad: ["", "a", "ab", "abc", "abbbb", "cbcbcbcb"],
},
pythonStringLiterals: {
  good: String.raw```
    ""
    'hello'
    "world"
    'a\'b'
    "a\"b"
    '\n'
    "a\tb"
    f'\u'
    """abc"""
    '''a''"''"'''
    """abc\xdef"""
    '''abc\$def'''
    '''abc\''''`
    .split("\n")
    .map((s) => s.trim()),
  bad: String.raw`
    'hello"
```

```
      "world'
      'a'b'
      "a"b"
      'a''
      "x""
      """""""""
      frr"abc"
      'a\'
      '''abc''''
      """`
      .split("\n")
      .map((s) => s.trim()),
  },
}

for (let name of Object.keys(testFixture)) {
  describe(`when matching ${name}`, () => {
    for (let s of testFixture[name].good) {
      it(`accepts ${s}`, () => {
        assert.ok(matches(name, s))
      })
    }
    for (let s of testFixture[name].bad) {
      it(`rejects ${s}`, () => {
        assert.ok(!matches(name, s))
      })
    }
  })
}
```

The file *regex_exercises.js* can be structured like so:

```
const regexes = {
  canadianPostalCode: /^ ........ $/,
  visa: /^ ........ $/,
  masterCard: /^ ........ $/,
  notThreeEndingInOO:  /^ ........ $/iu,
  divisibleBy16:  /^ ........ $/,
  eightThroughThirtyTwo:  /^ ........ $/,
  notPythonPycharmPyc:  /^ ........ $/u,
```

```
  restrictedFloats:  /^ ........ $/i,
  palindromes2358: /^ ........ $/,
  pythonStringLiterals: /^ ........ $/,
}

export function matches(name, string) {
  return regexes[name].test(string)
}
```

2. For the following C function:

```
int f(const int n) {
  return n % 2 == 0 ? n / 2 : 3 * n + 1;
}
```

   give both a highly optimized WebAssembly translation and a highly optimized x86-64 translation. You do not have to do this by hand; instead, use the Compiler Explorer, and set the optimization to `-O3`. Include comments in the translated code.

3. (Here's an exercise which logically belongs in the previous homework assignment, but sometimes it's best to space things out a bit.) Show the language $\{\langle M_1 \rangle \langle M_2 \rangle \mid L(M_1) = L(M_2)\}$ is undecidable, using a rigorous reduction argument.

For Your Project

Add optimizations to your compiler project. You do not have to do too many, but the few you do should be non-trivial. Because everyone in class is compiling a somewhat different language, you must check with me *early* regarding which optimizations are appropriate for your project. Don't worry, we will negotiate the set of optimizations that you are responsible for, so there should be no surprises when you get your grade.

Your project should end up with a project structured as follows:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json        -- (optional, you don't have to have one)
├── docs
```

```
|   └── ...                    -- logo, companion website, etc.
├── examples
|   └── ...                    -- lots of example programs
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js
|   ├── core.js
|   ├── analyzer.js
|   ├── optimizer.js           -- implement this for this assignment
|   └── generator.js
└── test
    ├── compiler.test.js
    ├── grammar.test.js
    ├── analyzer.test.js
    ├── optimizer.test.js      -- implement this for this assignment
    └── generator.test.js
```

The most important things to keep in mind as you wrap up your project are:

- 100% test coverage! Hold yourself to this goal. Be proud!
- Have your README and companion website completely up to date with your language. If there are parts of your language documented in the README or companion site but not yet implemented, you must mark your README or companion site with a "Not yet implemented" warning. If you have code in your compiler that is *not* documented in your README or companion site, that will result in a loss of points, as this is a situation that should not happen.
- Make sure there are no additional, unnecessary files in your repo. A pristine repo is a happy repo!
- Ensure all of the code has been Prettied. Your repo is public and you want to show your best face to the world!

Presentation!

During the last class session, **present, as your entire group, a lightning talk introducing (and showing off!) your language to the rest of the class**. Be personal, be funny, and be brief! You may use slides, video, and even music. If you are using slides, don't use too many, and make sure they are colorful...with images, *and please put these in your project repository*. If your language has a theme (pirates, horror, uwu, vikings, fashion, sci-fi, anime, social policy, medical, whatever), you can dress up (but avoid stereotypes and be respectful).

For tips on how to do a good lightning talk, see here, here, and here. What's a lightning talk? Wikipedia can tell you.

Grading Rubric

To help you get a good score on the remaining 80 points, here are all the things you will be graded on.

- Exercises (25 pts)
  - ✅ Exercise 1 (15 pts)
  - ✅ Exercise 2 (5 pts)
  - ✅ Exercise 3 (5 pts)
- Finished Compiler Project (45 pts)
  - ✅ Optimizer is strong
  - ✅ Project clones and runs without crashing
  - ✅ package.json is complete (author names, type=module, test script, etc.)
  - ✅ All tests pass
  - ✅ 100% test coverage
  - ✅ Command line operations that run the parser, the analyzer, and the generator separately all work
  - ✅ Can analyze, optimize, and generate code for Hello World
  - ✅ Can analyze, optimize, and generate code for a medium size program
  - ✅ Can analyze, optimize, and generate code for a really large program
  - ✅ Errors are detected and reported cleanly when run from the command line
  - ✅ Your language reached the point of being “complex enough.”
  - ✅ Code is nice, clean, well-organized, professional
  - ✅ Code has been prettied
  - ✅ Analyzer, optimizer, and code generator are all complete
  - ✅ README page is complete, useful, and accurate
  - ✅ Companion website is complete, useful, and accurate
  - ✅ README page OR companion website has a link to the grammar, making it easy to lookup
  - ✅ You have a lot of really good examples in your examples directory
  - ✅ Your logo looks good
  - ✅ Project commit log shows that you've been working all semester
  - ✅ Optional but strongly encouraged: You included something “fun” in the compiler—perhaps an easter egg (document the easter egg in the companion website)
- Presentation (30 pts)
  - ✅ You introduced yourselves
  - ✅ You introduced your language
  - ✅ You told the story of your language in a fun and engaging way
  - ✅ Slides or other visual aids are compelling
  - ✅ If you do have slides (you should) they show some interesting snippets of your language
  - ✅ The talk is not too short or too long

- ✅ The talk has some “structure” (prove to me you read the materials I linked to about how to make a
  lightning talk effective).
- ✅ Intro and outro were compelling, exciting, and memorable!

Also, as an audience member, you will be expected to be attentive, respectful, and encouraging, and celebrate and applaud your classmates when they present.

> ***Fairness in Team Grading***
>
> *Finally, I will be interviewing each team to gauge the contributions of each team member. The old ”I participated in meetings but we always committed from so-and-so’s computer” isn’t really acceptable.*